

|                              |                |            |
|------------------------------|----------------|------------|
| <b>Discrete Mathematics</b>  | Section        | 04         |
|                              | Student number | 22200622   |
| <b>Project – Spam filter</b> | Name           | Im, Ye Jin |

*If your explanation is less informative and insufficient, you may not get any points.  
Also, you should provide a discussion, otherwise you will get a penalty.*

● General information

| Item                                  | Your answer |
|---------------------------------------|-------------|
| The number of lines in your code.     | 281         |
| The number of functions in your code. | 5           |

● Functions

| Function name          | Function Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (bool) is_word         | Functions that remove non-alphabetic characters defined in specialChar, such as special characters or numbers.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| (void) csv_dictionary  | <p><i>Input_file</i> contains the contents of an email, and <i>output_dict</i> stores the words that occurred in the email and the number of times they occurred.</p> <ul style="list-style-type: none"> <li>- Save words and counts.</li> <li>- Separate mail content by spam(ham),"Subject:</li> <li>- Set word count to increment only once per mailing</li> </ul> <p>it reads the input file line by line, separates them by whitespace, and applies the is_word function.</p> <p>Unnecessary articles are removed because they are not meaningful for determining spam.</p> <p>If the word already exists, it is recorded in exist_words, and if it is a new word, it is incremented in word_cnt.</p> <p>Initialize exist_words every time when read a new mail.</p> <p>Save the words and counts to a csv file and close the file.</p> |
| (void) calculate_r     | <p>A function that takes the number of words stored in a dictionary and calculates the value of r(word) + compares it to a threshold and stores a flag.</p> <p>Calculates the r(word) value per word and compares the maximum r value to the threshold to determine if it is spam.</p> <p>If a word does not exist in the spam or ham lexicon, set its r value to 0.5.</p> <p>Single digit words are more likely to be spam, and etc words are more likely to be non-spam, so I added more of them to increase the success rate.</p>                                                                                                                                                                                                                                                                                                         |
| (void) load_dictionary | Function to read words and counts from a wordbook (.csv file) and store them in a map.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

|            |                                                                                                                                                                                                                                                                                                                                                                                              |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (int) main | <p>Set thresholds, specify path to train file (spam, ham), specify path to wordbook (spam, ham), specify file condensation to save r-values and save results.</p> <p>Run the csv_dictionary function to create the dictionaries and the load_dictionary function to get the word count from each.</p> <p>Finally, run the calculate_r function to save the r value and the final result.</p> |
|            |                                                                                                                                                                                                                                                                                                                                                                                              |
|            |                                                                                                                                                                                                                                                                                                                                                                                              |
|            |                                                                                                                                                                                                                                                                                                                                                                                              |
|            |                                                                                                                                                                                                                                                                                                                                                                                              |

- Screenshot of your program running

### Spam test20.csv result

```

test.cpp x
test.cpp > main()
255
256 int main() {
257     float threshold = 0.8; // 임계값
258     string spam_file = "emails/train/dataset_spam_train100.csv"; // spam_train 파일 입력
259     string spam_storage = "spam_dictionary.csv";
260     string ham_file = "emails/train/dataset_ham_train100.csv"; // ham_train 파일 입력
261     string ham_storage = "ham_dictionary.csv";
262     string test_file = "emails/test/dataset_spam_test20.csv"; // test.csv 파일 입력
263     string reject_storage = "reject.csv"; // reject.csv에 결과 저장
264     string result_storage = "result.csv";
265
266     csv_dictionary(spam_file, spam_storage); // spam 단어장 생성
267     csv_dictionary(ham_file, ham_storage); // ham 단어장 생성
268
269     // 단어장 로드
270     map<string, int> spam_dict;
271     map<string, int> ham_dict;
272
273     // spam_dict와 ham_dict에 단어와 개수를 저장
274     load_dictionary(spam_storage, spam_dict); // spam 단어장 파일을 spam_dict에 로드
275     load_dictionary(ham_storage, ham_dict); // ham 단어장 파일을 ham_dict에 로드
276
277     PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
278
279     (base) imagine@im-yejin-ui-MacBookPro DMproject#1 % g++ test.cpp -o test
280     (base) imagine@im-yejin-ui-MacBookPro DMproject#1 % ./test
281     ham
282     ham
283     ham
284     spam
285     spam
286     spam
287     spam
288     spam
289     spam
290     spam
291     ham
292     ham
293     spam
294     spam
295     spam
296     (base) imagine@im-yejin-ui-MacBookPro DMproject#1 %
  
```

### Ham test20.csv result

```

test.cpp x
test.cpp > main()
255
256 int main() {
257     float threshold = 0.8; // 임계값
258     string spam_file = "emails/train/dataset_spam_train100.csv"; // spam_train 파일 입력
259     string spam_storage = "spam_dictionary.csv";
260     string ham_file = "emails/train/dataset_ham_train100.csv"; // ham_train 파일 입력
261     string ham_storage = "ham_dictionary.csv";
262     string test_file = "emails/test/dataset_ham_test20.csv"; // test.csv 파일 입력
263     string reject_storage = "reject.csv"; // reject.csv에 결과 저장
264     string result_storage = "result.csv";
265
266     csv_dictionary(spam_file, spam_storage); // spam 단어장 생성
267     csv_dictionary(ham_file, ham_storage); // ham 단어장 생성
268
269     // 단어장 로드
270     map<string, int> spam_dict;
271     map<string, int> ham_dict;
272
273     // spam_dict와 ham_dict에 단어와 개수를 저장
274     load_dictionary(spam_storage, spam_dict); // spam 단어장 파일을 spam_dict에 로드
275     load_dictionary(ham_storage, ham_dict); // ham 단어장 파일을 ham_dict에 로드
276
277     PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
278
279     (base) imagine@im-yejin-ui-MacBookPro DMproject#1 % g++ test.cpp -o test
280     (base) imagine@im-yejin-ui-MacBookPro DMproject#1 % ./test
281     ham
282     ham
283     ham
284     ham
285     ham
286     ham
287     ham
288     ham
289     ham
290     ham
291     ham
292     ham
293     ham
294     ham
295     ham
296     (base) imagine@im-yejin-ui-MacBookPro DMproject#1 %
  
```

- Results

- Probability  $r(w_1, w_2, \dots, w_n)$  and predicted label (spam or non-spam) for different threshold (T) (0.6, 0.7, 0.8, 0.9 and 0.95)

| num<br>True  | True Label | probability<br>( $r$ ) | T = 0.6            | T = 0.7            | T = 0.8            | T = 0.9            | T = 0.95           |
|--------------|------------|------------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
|              |            |                        | predicted<br>label | predicted<br>label | predicted<br>label | predicted<br>label | predicted<br>label |
| s01          | Spam       | 0.794118               | spam               | spam               | ham                | ham                | ham                |
| s02          | Spam       | 0.745098               | spam               | spam               | ham                | ham                | ham                |
| s03          | Spam       | 0.794118               | spam               | spam               | ham                | ham                | ham                |
| s04          | Spam       | 0.928571               | spam               | spam               | spam               | spam               | ham                |
| s05          | Spam       | 0.909091               | spam               | spam               | spam               | spam               | ham                |
| s06          | Spam       | 0.916667               | spam               | spam               | spam               | spam               | ham                |
| s07          | Spam       | 0.909091               | spam               | spam               | spam               | spam               | ham                |
| s08          | Spam       | 0.95                   | spam               | spam               | spam               | spam               | spam               |
| s09          | Spam       | 0.909091               | spam               | spam               | spam               | spam               | ham                |
| s10          | Spam       | 0.909091               | spam               | spam               | spam               | spam               | ham                |
| s11          | Spam       | 0.837838               | spam               | spam               | spam               | ham                | ham                |
| s12          | Spam       | 0.875                  | spam               | spam               | spam               | ham                | ham                |
| s13          | Spam       | 0.863636               | spam               | spam               | spam               | ham                | ham                |
| s14          | Spam       | 0.837838               | spam               | spam               | spam               | ham                | ham                |
| s15          | Spam       | 0.5                    | ham                | ham                | ham                | ham                | ham                |
| s16          | Spam       | 0.75                   | spam               | spam               | ham                | ham                | ham                |
| s17          | Spam       | 0.928571               | spam               | spam               | spam               | spam               | ham                |
| s18          | Spam       | 0.9375                 | spam               | spam               | spam               | spam               | ham                |
| s19          | Spam       | 0.837838               | spam               | spam               | spam               | ham                | ham                |
| s20          | Spam       | 0.909091               | spam               | spam               | spam               | spam               | ham                |
| h01          | Non-spam   | 0.8                    | ham                | ham                | ham                | ham                | ham                |
| h02          | Non-spam   | 0.857143               | ham                | ham                | ham                | ham                | ham                |
| h03          | Non-spam   | 0.8                    | ham                | ham                | ham                | ham                | ham                |
| h04          | Non-spam   | 0.794118               | ham                | ham                | ham                | ham                | ham                |
| h05          | Non-spam   | 0.537037               | ham                | ham                | ham                | ham                | ham                |
| h06          | Non-spam   | 0.611111               | ham                | ham                | ham                | ham                | ham                |
| h07          | Non-spam   | 0.857143               | ham                | ham                | ham                | ham                | ham                |
| h08          | Non-spam   | 0.833333               | ham                | ham                | ham                | ham                | ham                |
| h09          | Non-spam   | 0.875                  | ham                | ham                | ham                | ham                | ham                |
| h10          | Non-spam   | 0.875                  | ham                | ham                | ham                | ham                | ham                |
| h11          | Non-spam   | 0.9                    | ham                | ham                | ham                | ham                | ham                |
| h12          | Non-spam   | 0.875                  | ham                | ham                | ham                | ham                | ham                |
| h13          | Non-spam   | 0.863636               | ham                | ham                | ham                | ham                | ham                |
| h14          | Non-spam   | 0.5                    | ham                | ham                | ham                | ham                | ham                |
| h15          | Non-spam   | 0.916667               | ham                | ham                | ham                | ham                | ham                |
| h16          | Non-spam   | 0.8                    | ham                | ham                | ham                | ham                | ham                |
| h17          | Non-spam   | 0.875                  | ham                | ham                | ham                | ham                | ham                |
| h18          | Non-spam   | 0.5                    | ham                | ham                | ham                | ham                | ham                |
| h19          | Non-spam   | 0.8                    | ham                | ham                | ham                | ham                | ham                |
| h20          | Non-spam   | 0.916667               | ham                | ham                | ham                | ham                | ham                |
| Accuracy (%) |            |                        | 97.5               | 97.5               | 87.5               | 75                 | 52.5               |

※ Accuracy is calculated from the following equation.

$$\text{Accuracy (\%)} = \frac{\text{Number of correctly predicted emails}}{40} \times 100$$

- Discussion (your interpretation of the results and possible strategy to improve the algorithm)

You can see that the smaller the threshold value, the higher the accuracy. It is often seen that even when  $\text{probability}(r)$  is measured higher than the threshold, it comes out as HAM. In this case, we saw that there were a lot of etc in the train ham mail, while there were very few etc in the train\_spam\_mail. Therefore, we added a variable called will\_ham to increase the value of the will\_ham variable when etc appears frequently in the mail, so that if the probability is higher than the threshold, it is judged as ham.

The algorithm will be more accurate if we further develop it to recognize specific keywords or word types that appear frequently in spam. Also, although there are only 100 train data for each spam/ham mail, it is expected that higher accuracy can be achieved by increasing the number of train data.

#### □ Codes

// Put code here and submit your original executable C/C++ code.  
// If C/C++ code does not run, then No point will be given.

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <regex>
#include <map>
#include <unordered_set>

using namespace std;
typedef map<string, int>::iterator Map;

// 특수문자 삭제
bool is_word(string& word) {
    regex specialChar("[^a-zA-Z]");
    word = regex_replace(word, specialChar, "");
    return !word.empty();
}

void csv_dictionary(string& file, string& storage) {
    ifstream input_file(file);
    ofstream output_dict(storage);

    map<string, int> word_cnt; // 단어가 등장한 메일 개수 저장
    string line;
    bool in_spam_subject = false;
    unordered_set<string> exist_words;

    //디버깅
    if (!input_file.is_open()) {
        cout << "Failed to open input file!" << endl;
        return;
    }
    if (!output_dict.is_open()) {
        cout << "Failed to open output file!" << endl;
        return;
    }

    //파일 읽기 실행
    while (getline(input_file, line)) {
```

```

stringstream ss(line);
string word;

//spamSubject 기준으로 메일 하나를 구분
if ((line.find("spam,\"Subject:\") != string::npos) || (line.find("ham,\"Subject:\") != string::npos)) {
    if (in_spam_subject) { // 새 spamSubject 일 경우 (-> 새로운 메일)
        exist_words.clear();
    }
    in_spam_subject = true;
}

//단어 추적
if (in_spam_subject) {
    while (ss >> word) {
        if (is_word(word) &&
            word != "numlabeltext" && word != "spamSubject" && word != "hamSubject" &&
            word != "a" && word != "the" && word != "in" && word != "by" &&
            word != "of" && word != "and" && word != "is" && word != "are" &&
            word != "as" && word != "at" && word != "be" && word != "an" &&
            word != "for" && word != "from" && word != "has" && word != "have") {

            if (exist_words.find(word) == exist_words.end()) {
                word_cnt[word]++; // 새로운 단어가 등장하면 개수 증가 (중복단어 무시)
                exist_words.insert(word);
            }
        }
    }
}

//csv 에 저장 (단어 | 개수)
for (Map it = word_cnt.begin(); it != word_cnt.end(); ++it) {
    output_dict << it->first << "," << it->second << endl;
}

input_file.close();
output_dict.close();
}

void calculate_r(string& test_file, map<string, int>& spam_dict, map<string,
    int>& ham_dict, string& reject_storage, float threshold, string& result_storage) {

```

```

ifstream input_file(test_file);
ofstream reject_file(reject_storage);
ofstream result_file(result_storage);

string line;

float max_r = 0.0; // 각 행에 대한 최대 r_value

int will_spam = 0; // 한자리 단어 나올때마다 증가 (한자리 단어 많이 나올수록 스팸일 확률 증가)
int will_ham = 0; // etc 단어 나올때마다 증가 (etc 단어 나올수록 스팸 아닐 확률 증가)

reject_file << "word,r(word)" << endl;

bool subject_found = false; // Subject 가 처음 발견되면 이후부터 텍스트를 처리하도록 하는 플래그
string current_text = ""; // 현재까지 읽은 텍스트 (한 이메일의 내용)

while (getline(input_file, line)) {
    // 맨 처음 spam,"Subject:" 구간이 나오기 전까지의 내용은 무시
    if (!subject_found && (line.find("spam,\"Subject:\") != string::npos || line.find("ham,\"Subject:\") != string::npos)) {
        subject_found = true;
        current_text = "";
        continue;
    }

    if (subject_found) {
        // spamSubject 발견 시 그 전에 저장된 내용을 처리
        if ((line.find("spam,\"Subject:\") != string::npos) || (line.find("ham,\"Subject:\") != string::npos)) {
            if (!current_text.empty()) {
                // r(word) 계산
                float max_r = 0.0;

                stringstream ss(current_text);
                string word;

                // 단어별 r(word) 계산
                while (ss >> word) {
                    if (is_word(word)) {
                        transform(word.begin(), word.end(), word.begin(), ::tolower); // 대소문자 구분 X
                        int spam_count, ham_count;

                        if (spam_dict.find(word) != spam_dict.end()) {
                            spam_count = spam_dict.at(word);
                        }
                        else
                            spam_count = 0;

```



```

        if (ham_dict.find(word) != ham_dict.end()) {
            ham_count = ham_dict.at(word);
        }
        else
            ham_count = 0;

        // p(word)와 q(word) 계산 (둘 중에 하나가 없을 경우 확률은 0.5 로 지정)
        double p = spam_count / 100.0;
        double q = ham_count / 100.0;
        double r;
        if (p == 0 || q == 0)
            r = 0.5;
        else if (p + q > 0){
            r = p / (p + q);
        }

        // r(word) 값 저장
        reject_file << word << ", " << r << endl;

        if (r > max_r) {
            max_r = r;
        }

        // 정확도 높이기 위한 추가 설정
        if (word.length() == 1) { // 1 자리 단어는 will_spam 값을 증가
            will_spam++;
        }
        if (word == "ect") { // 1 자리 단어는 will_spam 값을 증가
            will_ham++;
        }
    }
}

// 최대 r(word)와 threshold 비교
if ((max_r >= threshold || will_spam == 1) && will_ham < 2) {
    result_file << max_r << ",spam" << endl;
    cout << "spam" << endl;
} else{
    result_file << max_r << ",ham" << endl;
    cout << "ham" << endl;
}
}

```

```

    }

    // 새 메일내용 시작
    current_text = "";
}

// 내용 누적
current_text += " " + line;
}
}

// 마지막 내용 처리
if (!current_text.empty()) {
    float max_r = 0.0;
    stringstream ss(current_text);
    string word;

    while (ss >> word) {
        if (is_word(word)) {
            transform(word.begin(), word.end(), word.begin(), ::tolower);
            int spam_count, ham_count;

            if (spam_dict.find(word) != spam_dict.end()) {
                spam_count = spam_dict.at(word);
            }
            else
                spam_count = 0;

            if (ham_dict.find(word) != ham_dict.end()) {
                ham_count = ham_dict.at(word);
            }
            else
                ham_count = 0;

            double p = spam_count / 100.0;
            double q = ham_count / 100.0;
            double r;
            if (p == 0 || q == 0)
                r = 0.5;
            else if (p + q > 0){
                r = p / (p + q);
            }
        }
    }
}

```

```

    }

    reject_file << word << ", " << r << endl;
    if (r > max_r) {
        max_r = r;
    }

    if (word.length() == 1) {
        will_spam++;
    }
    if (word == "ect") {
        will_ham++;
    }
}

}

if ((max_r >= threshold || will_spam == 1) && will_ham < 2) {
    result_file << max_r << ",spam" << endl;
    cout << "spam" << endl;
} else {
    result_file << max_r << ",ham" << endl;
    cout << "ham" << endl;
}

}

// 파일 닫기
input_file.close();
reject_file.close();
}

void load_dictionary(string& file, map<string, int>& dict) {
    ifstream input_file(file);
    string word;
    int count;

    // 디버깅
    if (!input_file.is_open()) {
        cout << "Error opening file: " << file << endl;
        return;
    }
}

```

```

// 파일에서 단어, 개수 읽기
while (getline(input_file, word, ',') && input_file >> count) {
    transform(word.begin(), word.end(), word.begin(), ::tolower); //대소문자 구분 X

    dict[word] = count; // map 에 단어, 개수 추가
    input_file.ignore(numeric_limits<streamsize>::max(), '\n'); // 줄바꿈 문자 처리
}

input_file.close();
}

int main() {
    float threshold = 0.95; // 임계값
    string spam_file = "emails/train/dataset_spam_train100.csv"; // spam_train 파일 입력
    string spam_storage = "spam_dictionary.csv";
    string ham_file = "emails/train/dataset_ham_train100.csv"; // ham_train 파일 입력
    string ham_storage = "ham_dictionary.csv";
    string test_file = "emails/test/dataset_ham_test20.csv"; // test.csv 파일 입력
    string reject_storage = "reject.csv"; // reject.csv 에 결과 저장
    string result_storage = "result.csv";

    csv_dictionary(spam_file, spam_storage); // spam 단어장 생성
    csv_dictionary(ham_file, ham_storage); // ham 단어장 생성

    // 단어장 로드
    map<string, int> spam_dict;
    map<string, int> ham_dict;

    // spam_dict 와 ham_dict 에 단어와 개수를 저장
    load_dictionary(spam_storage, spam_dict); // spam 단어장 파일을 spam_dict 에 로드
    load_dictionary(ham_storage, ham_dict); // ham 단어장 파일을 ham_dict 에 로드

    // r_values 계산 및 reject.csv, result.csv 저장
    calculate_r(test_file, spam_dict, ham_dict, reject_storage, threshold, result_storage);

    return 0;
}

```